

Reproducibility guide

Repository source: REPRODUCIBILITY.md

Contents

Continuous-integration environments	1
Recorded successful clean clone	1
Fresh-clone procedure	1
1. Python checks	2
2. R checks	2
3. Manuscript build	2
4. Markdown-derived PDFs	2
5. Metadata and PDF preflight	3
What is exact and what is numerical	3
PDF reproducibility	3
GitHub Actions	3
Release-candidate reproduction	3
Reporting a discrepancy	4

Release: v0.1.0 (September 6, 2026)

Canonical repository: <https://github.com/swihart/minvol-degree3-spectral-bound>

Versioned release: <https://github.com/swihart/minvol-degree3-spectral-bound/releases/tag/v0.1.0>

This guide explains how to reproduce the symbolic checks, numerical cross-checks, manuscript build, and typeset Markdown documentation from a fresh clone.

Continuous-integration environments

The GitHub Actions workflow uses:

- Ubuntu 24.04;
- Python 3.13;
- the exact Python package versions in `requirements.txt`;
- R 4.6.1, using base R only;
- Pandoc 3.10.1;
- XeLaTeX, `latexmk`, and the TeX packages installed by the workflow; and
- `pdfinfo`, `pdffonts`, and `pdftotext` from Poppler for PDF preflight.

The workflow file is `.github/workflows/verification.yml`.

Recorded successful clean clone

A clean-clone reproduction was completed on 2026-09-04 for commit `96cea57275070f21babf30477ab1b128ec0f5eb6`. The isolated clone ran the Python and R suites, rebuilt the manuscript and every Markdown-derived PDF, passed the metadata and local checksum checks, and remained byte-clean afterward. The full Poppler PDF preflight was delegated to the three-job GitHub Actions run, which was green for the same commit.

The complete environment and interpretation are recorded in [CLEAN_CLONE_CHECK.md](#).

Fresh-clone procedure

To reproduce the fixed v0.1.0 release, clone the tag and enter the repository:

```
git clone --branch v0.1.0 --depth 1 \
  https://github.com/swihart/minvol-degree3-spectral-bound.git
cd minvol-degree3-spectral-bound
```

To inspect later development instead, clone the default branch without the `--branch v0.1.0` option. Results should always be reported with the exact tag or commit used.

1. Python checks

Create and activate a virtual environment if desired, then install the pinned requirements and run the suite:

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
./run_python_checks.sh
```

A successful run ends with all of the following:

```
EXACT DEGREE-3 IDENTITY CHECK: PASS
ALTERNATIVE COMPACT-FORM CHECK: PASS
PAIRING-COUNT CHECK: PASS
BOUND ARITHMETIC CHECK: PASS
NUMERICAL STRESS TESTS: PASS
```

2. R checks

The R scripts use base R only:

```
./run_r_checks.sh
```

A successful run contains:

```
R PAIRING-COUNT CHECK: PASS
R NUMERICAL STRESS TESTS: PASS
```

The runner records both outputs and the R session information in `expected/`.

3. Manuscript build

With Python 3, `latexmk`, and a standard LaTeX installation available:

```
./build_paper.sh
```

The resulting manuscript is:

```
paper/degree3_spectral_refinement.pdf
```

4. Markdown-derived PDFs

With Pandoc, Python 3, and XeLaTeX available:

```
./build_markdown_pdfs.sh
```

The rendered files are written beneath `rendered/markdown/`, preserving the source directory structure. Their SHA-256 hashes are written to:

```
rendered/markdown/SHA256SUMS.txt
```

On macOS, verify the tracked files with:

```
shasum -a 256 -c rendered/markdown/SHA256SUMS.txt
```

On systems with GNU coreutils, use:

```
sha256sum -c rendered/markdown/SHA256SUMS.txt
```

5. Metadata and PDF preflight

Check consistency of the author name, AI-system identification, citation file, licensing declaration, release version, release date, and public URLs:

```
./ci/preflight_metadata.sh  
./ci/preflight_release.sh
```

With Poppler installed, validate the manuscript and every Markdown-derived PDF:

```
./ci/preflight_pdfs.sh
```

The PDF check confirms that each PDF is nonempty and readable, has at least one page, contains extractable text, uses embedded fonts, and has a valid checksum when applicable. It also confirms that the manuscript contains the full author name, candidate coefficient, AI-assistance disclosure, and model designation.

What is exact and what is numerical

The general degree-3 tensor identities are checked with exact symbolic arithmetic by the Python/SymPy programs. The R programs are independently written numerical and finite-enumeration cross-checks. Agreement between the two languages helps detect implementation and convention errors, but it is not a substitute for mathematical review of the proof.

PDF reproducibility

The build scripts fix the source-date epoch. For Markdown-derived PDFs, the XeLaTeX driver is asked to disable stream compression and PDF object streams; the postprocessor then canonicalizes embedded-font subset prefixes, metadata dates, and the PDF trailer identifier. A lightweight regression test in `ci/check_pdf_canonicalizer.sh` verifies those canonicalizations. The lack of compression makes the documentation PDFs somewhat larger, but makes volatile font names visible to deterministic postprocessing.

Repeated builds with an unchanged toolchain are intended to be byte-for-byte reproducible. Different operating systems, TeX Live releases, fonts, or Pandoc versions may still produce visually equivalent but byte-different PDFs. In the recorded 2026-09-04 clean-clone run, however, macOS with Pandoc 2.19.2 and TeX Live 2022 reproduced all tracked outputs byte for byte. The Markdown and LaTeX files are the editable source files; the tracked PDFs are rendered counterparts provided for convenient reading.

GitHub Actions

The workflow runs automatically on pushes to `main`, pushes of version tags, pull requests, and manual requests from the Actions tab. It stores the Python and R logs and the generated PDFs as temporary workflow artifacts for inspection. A green run means that the repository's checks and builds completed in the declared environments; it is not an independent proof review.

Release-candidate reproduction

Before the `v0.1.0` tag is created, the exact release-candidate commit is required to pass:

1. all three GitHub Actions jobs on `main`;
2. the local clean-clone helper against `origin/main`;
3. the release metadata preflight; and
4. a final check that rebuilding leaves the tracked checkout clean.

The tag must point to the same commit that passed these checks, with no tracked change in between. Pushing the tag starts the same three-job workflow for the tagged ref; that run must also pass before the GitHub release is published. The release page should record the commit hash and the final clean-clone result.

Reporting a discrepancy

Record the following when reporting a failed or differing run:

- commit hash;
- operating system and architecture;
- Python, SymPy, NumPy, R, Pandoc, and TeX versions;
- the full command used; and
- the complete console output beginning at the first warning or error.